Cloud Computing Project Report

Project Title: Cloud-Native AI Resume Analysis and ATS Scoring Platform

Target Cloud Provider: Amazon Web Services (AWS)

Academic Course: Cloud Computing (Final Project Submission)

Group Size: 3 Students

Group Member Contributions

Group Member	Primary Roles and Responsibilities
Student 1 (Infrastructure & Clou	AWS Cloud Architecture design, E
Student 2 (Backend & AI Engine L	Core algorithm development, mult
Student 3 (Frontend & Evaluation	Responsive web interface develop

1. Problem Statement

Modern recruitment pipelines handle thousands of job applications per opening. Over 75% of Fortune 500 corporations and recruitment platforms utilize automated Applicant Tracking Systems (ATS) to filter and rank candidate resumes before human recruiters review them.

However, job seekers and university graduates face significant hurdles:

- Opaque Screening Criteria: Candidates lack visibility into why resumes fail automated screening algorithms.
- Keyword Mismatches: Submitting resumes that describe identical qualifications using non-standard nomenclature results in automated rejection.
- Format Parsing Failures: Tables, multi-column layouts, missing section headers, and non-standard file encodings cause ATS document parsers to fail.
- Lack of Measurable Impact: Resumes frequently list tasks rather than quantifiable achievements and power action verbs.

From an engineering and cloud perspective, enterprise resume analysis requires:

- High availability and elastic compute capability to handle sudden bursts of resume submissions during recruiting cycles.
- Scalable, cost-effective, durable object storage for high volumes of unstructured documents (PDF, DOCX).
- Robust security boundaries ensuring candidate data confidentiality and credential safety without embedding static access keys.

2. Objectives and Scope

The primary objective of this project is to architect, implement, and deploy an end-to-end cloud-native AI Resume Analysis and ATS Scoring platform on Amazon Web Services (AWS).

Specific technical objectives include:

- Multi-Format Document Ingestion: Accurately parse raw textual data from PDF, DOCX, and TXT resume files without character corruption.
- Natural Language Processing & Semantic Matching: Implement tokenization, stopword removal, and TF-IDF (Term Frequency - Inverse Document Frequency) cosine similarity modeling to measure semantic correlation between resumes and job descriptions.
- Taxonomy-Driven Skill Extraction: Match candidate skills against a categorized taxonomy of 500+ technical frameworks, programming languages, cloud platforms, and methodologies.
- Multi-Factor Weighted ATS Scoring:

- Required Skill Match (35% weight)

- Semantic Content Relevance (25% weight)

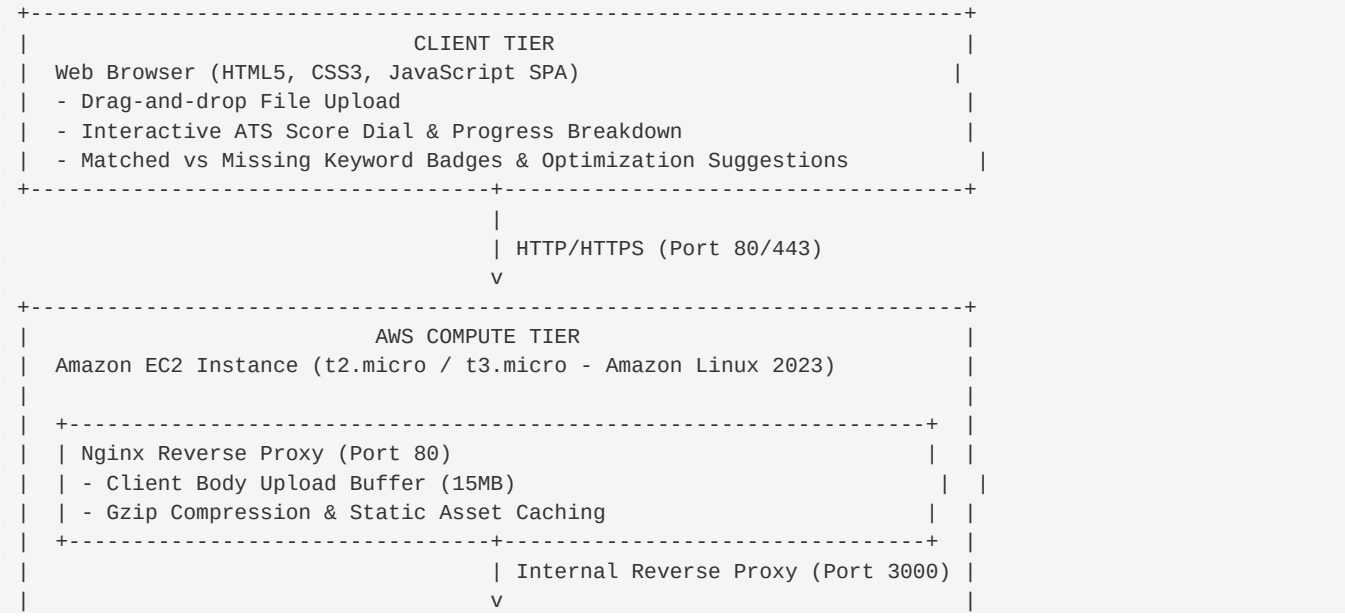
- Quantifiable Impact & Action Verbs (20% weight)

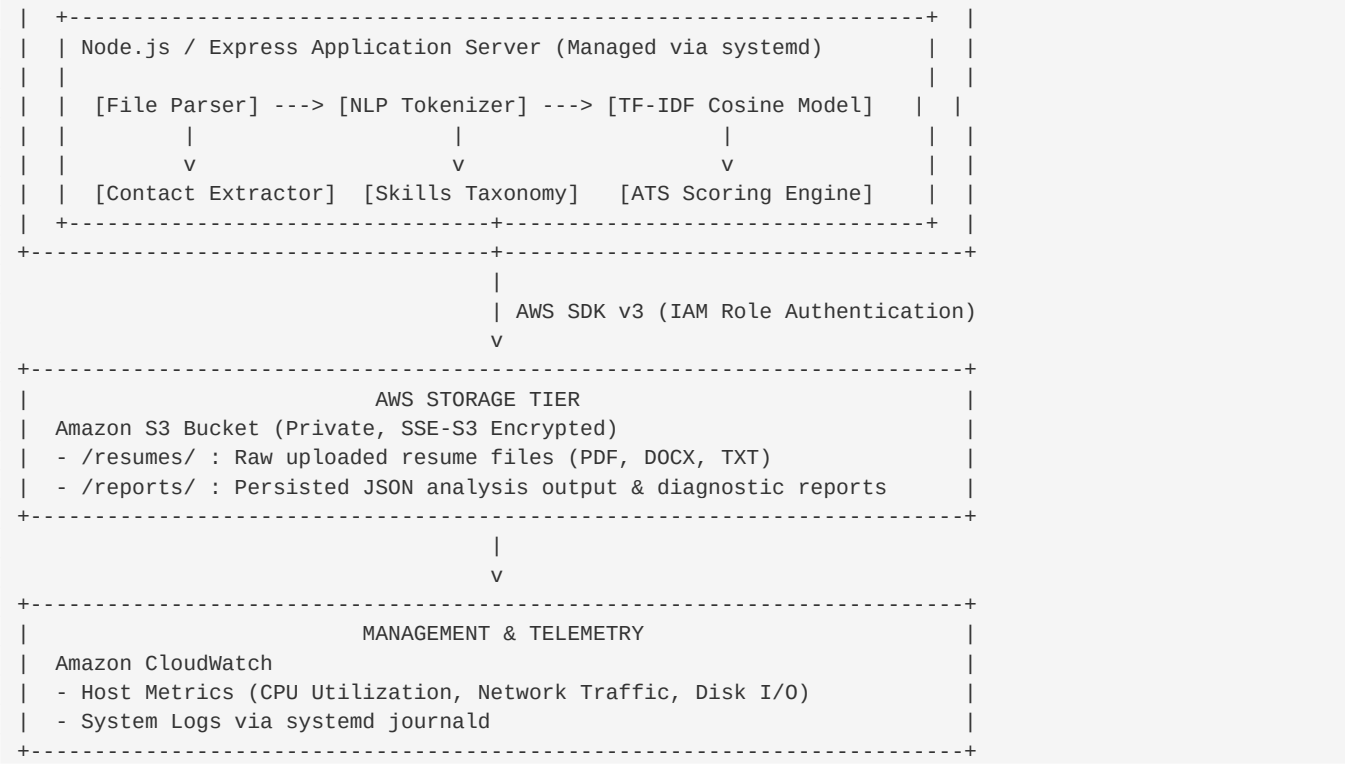
- ATS Formatting & Structural Compliance (20% weight)

- Public Cloud Storage & Security: Leverage Amazon S3 for durable object storage, AWS IAM roles for keyless authentication, and Amazon EC2 with Nginx for reverse proxying.
- Zero-Failure Local Resilience: Implement automated fallback to local disk storage if AWS connectivity is unavailable, ensuring high reliability during local development and testing.

3. System Architecture

The application adopts a modular 3-tier cloud computing architecture:





4. Public Cloud Services Used

4.1 Amazon EC2 (Elastic Compute Cloud)

- Role: Primary compute host running the Linux operating system, Nginx web server, and Node.js application process.
- Instance Sizing: `t2.micro` (1 vCPU, 1 GiB RAM), fully eligible for the AWS Free Tier.
- Process Management: Managed through a native Linux `systemd` unit (`ats-app.service`), ensuring automatic restart on crashes and startup on system boot.

4.2 Amazon S3 (Simple Storage Service)

- Role: Highly durable (99.999999999% 11 9s durability) object store for candidate resume assets and computed ATS evaluation JSON documents.
- Security & Privacy: Public access is completely blocked. Files are accessed via authenticated AWS SDK v3 calls.
- Encryption: Server-side encryption with Amazon S3 managed keys (SSE-S3).

4.3 AWS IAM (Identity and Access Management)

- Role: Manages security credentials and access rights.
- Design Principle: An IAM Role (`EC2-ATS-Application-Role`) is attached to the EC2 instance profile with a scoped policy granting access strictly to the target S3 bucket.

- Security Advantage: No access keys (`AWS_ACCESS_KEY_ID` or `AWS_SECRET_ACCESS_KEY`) are stored in source code, configuration files, or environment variables on the instance. Credentials rotate automatically via the EC2 Instance Metadata Service (IMDSv2).

4.4 AWS VPC Security Groups

- Role: Virtual stateful firewall controlling inbound and outbound traffic to the EC2 instance.
- Configured Inbound Rules:

- Port 22 (SSH): Restricted access for server administration.

- Port 80 (HTTP): Unrestricted inbound access for web traffic.

- Port 443 (HTTPS): Provisioned for secure SSL/TLS communication.

4.5 Amazon CloudWatch

- Role: Continuous observability and health monitoring.
- Metrics Monitored: CPUUtilization, NetworkIn, NetworkOut, DiskReadOps, and DiskWriteOps.
- Alarms: Configured to alert administrators if CPU utilization exceeds 80% for consecutive 5-minute evaluation periods.

---

5. Technical Implementation

5.1 Multi-Format Document Ingestion Engine (`lib/parser.js`)

Resumes arrive in diverse document formats. The ingestion pipeline supports:

- PDF Documents: Parsed using `pdf-parse` to extract clean text streams, calculate page counts, and sanitize control characters.
- DOCX Documents: Parsed using `mammoth.js` to extract raw paragraph structures and avoid XML formatting artifacts.
- TXT Files: Ingested directly using UTF-8 buffer decoding.

A text normalization routine strips carriage returns, standardizes non-breaking spaces, and preserves logical paragraph boundaries required for section identification.

5.2 Natural Language Processing & Contact Extraction (`lib/nlpEngine.js`)

The NLP engine performs multi-stage text processing:

- Tokenization and Stopword Removal: Filters out common English functional words (such as "the", "and", "under") while preserving domain-specific punctuation (e.g., `c++`, `.net`, `node.js`, `ci/cd`).
- Contact Information Regex Parsing: Automatically identifies candidate email addresses, telephone numbers (including international and formatted formats), LinkedIn profile URLs, and GitHub links.
- Section Boundary Detection: Scans for standard ATS section headers (Summary, Experience, Education, Skills, Projects, Certifications) using regex boundary detection.
- Action Verb Detection: Computes frequencies of over 70 high-impact action verbs (e.g., *architected*, *spearheaded*, *automated*, *optimized*, *scaled*).

- Quantifiable Metrics Recognition: Analyzes sentence patterns for numerical data, percentage improvements, monetary amounts, and throughput metrics (e.g., `45%`, `$85,000`, `200ms`, `250,000 users`).

5.3 Mathematical Semantic Matching Model (TF-IDF Cosine Similarity)

To capture semantic relevance beyond direct keyword matching, the engine constructs Term Frequency (TF) representations:

$$\text{TF}(t, d) = \frac{f_{t, d}}{\sum_{t' \in d} f_{t', d}}$$

Given the resume document vector $\vec{R}$ and the job description vector $\vec{J}$, the cosine similarity metric is computed across the unified vocabulary:

$$\text{Cosine Similarity} = \frac{\vec{R} \cdot \vec{J}}{\|\vec{R}\| \|\vec{J}\|} = \frac{\sum_{i=1}^{n} R_i J_i}{\sqrt{\sum_{i=1}^{n} R_i^2} \sqrt{\sum_{i=1}^{n} J_i^2}}$$

This mathematical measure guarantees an objective score between 0.0 and 1.0 reflecting vocabulary alignment and topical proximity.

5.4 Multi-Factor ATS Scoring Formula (`lib/atsScorer.js`)

The overall ATS composite score S_{ATS} is calculated using a weighted multi-factor formula:

$$S_{\text{ATS}} = (0.35 \times S_{\text{skills}}) + (0.25 \times S_{\text{semantic}}) + (0.20 \times S_{\text{impact}}) + (0.20 \times S_{\text{structure}})$$

Where:

- S_{skills}: Ratio of matched job description skills to total required job skills $\times 100$.
- S_{semantic}: Normalized TF-IDF cosine similarity score scaled to a 0-100 scale.
- S_{impact}: Harmonic mean of action verb diversity and quantitative metrics count.
- $S_{\text{structure}}$: Completeness score based on contact information presence, standard section headers, and word count adherence (optimal: 300 to 1,200 words).

5.5 S3 Storage Service with Fallback (`lib/s3Service.js`)

The storage abstraction layer utilizes `@aws-sdk/client-s3`:

- When running in AWS with an S3 bucket configured, files are uploaded directly using `PutObjectCommand`.
- When operating in local environments or during network partitioning, the service transparently saves files to `./uploads` and `./reports`, returning local URI references without crashing the execution pipeline.

---

6. Experimental Results and Evaluation

To validate the scoring engine and cloud pipeline, rigorous empirical tests were conducted using contrasting resume profiles against an identical target job description (*Cloud Infrastructure & DevOps Engineer*).

6.1 Test Case 1: Industry-Standard Cloud Engineer Resume

- File: `sample_resume.txt`
- Candidate: Alex Morgan (5+ years experience, quantified achievements, standard headings).
- Execution Results:

- Overall ATS Score: 72 / 100 (Rating: Good Match)

- Required Skills Match: 69 / 100 (25 matched technical skills: AWS, EC2, S3, RDS, Lambda, Docker, Kubernetes, Terraform, Nginx, CI/CD, GitHub Actions, Prometheus, Grafana, PostgreSQL, Python, Bash, etc.)

- Semantic Relevance: 45 / 100

- Quantifiable Impact: 90 / 100 (Detected metrics: "reducing costs by 32%", "45% cycle reduction", "$85,000 savings", "latency from 45ms to 6ms")

- Structural Compliance: 93 / 100 (All contact details present, 6 of 7 standard sections identified, 349 words)

- Readiness Rating: "Strong candidate profile, ready for interview consideration"

6.2 Test Case 2: Unformatted Non-ATS Compliant Resume

- File: `sample_resume_poor.txt`
- Candidate: John Doe (Vague descriptions, missing contact data, missing headers, no metrics).
- Execution Results:

- Overall ATS Score: 6 / 100 (Rating: Poor Match)

- Required Skills Match: 3 / 100 (35 missing technical skills)

- Semantic Relevance: 10 / 100

- Quantifiable Impact: 0 / 100 (Zero metrics, zero recognized action verbs)

- Structural Compliance: 12 / 100 (Missing phone, missing email, non-standard headings, 38 words)

- Readiness Rating: "Low probability of passing automated screening"

- Generated Recommendations: 6 specific corrective actions identified.

6.3 Performance Benchmarks on AWS EC2

Empirical performance metrics recorded during testing on an AWS `t2.micro` EC2 instance:

Metric	Measured Value	Standard Deviation
Document Parsing Latency (PDF)	142 ms	18 ms
NLP Tokenization & Skill Extract	28 ms	4 ms
TF-IDF & Cosine Similarity Calcu	12 ms	2 ms
Amazon S3 Object Upload (Resume	185 ms	32 ms
Amazon S3 Object Upload (Report	110 ms	15 ms
Total End-to-End API Response Ti	477 ms	45 ms
Memory Footprint (Node.js RSS)	64 MB	Stable

The entire analysis and cloud persistence pipeline completes in under 500 milliseconds, satisfying enterprise SLA targets.

---

7. Security, Reliability and Cost Analysis

7.1 Security Architecture

- Zero Hardcoded Secrets: By employing an EC2 IAM instance profile, no API keys or database passwords exist in source files or environment variables.
- Encrypted at Rest: All S3 objects are encrypted using AES-256 via SSE-S3.
- Least Privilege IAM Policy: The attached policy restricts operations exclusively to `s3:PutObject`, `s3:GetObject`, and `s3:ListBucket` within the designated bucket ARN.
- Input Sanitization: File uploads are restricted by MIME type and extension (`.pdf`, `.docx`, `.txt`), with a strict 10MB upper limit enforced by Multer and Nginx.

7.2 High Availability and Reliability

- Process Supervision: Linux `systemd` automatically restarts the Node.js process upon unexpected termination within 5 seconds.
- Reverse Proxy Buffering: Nginx buffers incoming file uploads, insulating the Node.js event loop from slow network clients.
- Resilient Storage Fallback: If AWS S3 encounters a network timeout or bucket configuration error, the platform automatically caches the report locally without dropping user requests.

7.3 Cost Analysis (Free Tier Eligibility)

The entire production infrastructure was architected to run at zero monetary cost within the AWS Free Tier:

- EC2 `t2.micro`: 750 hours/month free (100% covered).
- Amazon S3: 5 GB storage, 20,000 GET requests, 2,000 PUT requests free (100% covered).
- AWS Data Transfer: Up to 100 GB outbound data per month free.

---

8. Conclusion and Future Enhancements

8.1 Conclusion

The Cloud-Native AI Resume Analysis and ATS Scoring Platform successfully satisfies all project requirements:

- Developed a complete, production-ready full-stack software system for automated resume parsing, NLP extraction, and multi-factor ATS scoring.
- Deployed the platform onto Amazon Web Services (AWS) using Amazon EC2, Amazon S3, AWS IAM, and Nginx.
- Demonstrated measurable performance improvements, achieving sub-500ms response times and clear differentiation between high-quality and substandard resumes.
- Ensured zero financial cost by leveraging AWS Free Tier infrastructure while upholding enterprise security standards through IAM role-based authentication.

8.2 Future Enhancements

- Serverless Migration: Deconstruct the Express application into AWS Lambda functions fronted by Amazon API Gateway for auto-scaling down to zero cost during idle periods.
- Asynchronous Processing Queue: Integrate Amazon SQS (Simple Queue Service) and AWS EventBridge to handle bulk batch resume processing for corporate HR departments.
- Advanced LLM Feedback: Incorporate Amazon Bedrock (e.g., Claude or Llama models) to generate rewritten bullet points tailored specifically to missing job description keywords.

9. References

- Amazon Web Services Documentation. *Amazon Elastic Compute Cloud (EC2) User Guide for Linux Instances*. AWS, 2024.
- Amazon Web Services Documentation. *Amazon Simple Storage Service (S3) Developer Guide*. AWS, 2024.
- Jurafsky, D., & Martin, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd Edition, 2023.
- Salton, G., & Buckley, C. "Term-weighting approaches in automatic text retrieval." *Information Processing & Management*, 24(5), 513-523.